

Factory

08 June 2026 03:46 PM

Factory Design Pattern is used to create objects without exposing the object creation logic to the client and to make the client independent of concrete classes.

Types of Factory:

1. Simple factory
2. Abstract factory
3. Factory Method

1.Simple factory : A **Simple Factory** is not an official GoF design pattern, but it is a very common pattern used to centralize object creation.

Eg:

using System;

```
// --- Burger Interface ---
public interface IBurger
{
    void Prepare();
}

// --- Concrete Burger Implementations ---
public class BasicBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Basic Burger with bun, patty, and ketchup!");
    }
}

public class StandardBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Standard Burger with bun, patty, cheese, and lettuce!");
    }
}

public class PremiumBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Premium Burger with gourmet bun, premium patty, cheese, lettuce, and secret sauce!");
    }
}

// --- Burger Factory ---
public class BurgerFactory
{
    public IBurger CreateBurger(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
        {
            return new BasicBurger();
        }
        else if (type.Equals("standard", StringComparison.OrdinalIgnoreCase))
        {
            return new StandardBurger();
        }
        else if (type.Equals("premium", StringComparison.OrdinalIgnoreCase))
        {
            return new PremiumBurger();
        }

        Console.WriteLine("Invalid burger type!");
        return null;
    }
}
```

```
// --- Main Program ---
class Program
{
    static void Main(string[] args)
    {
        string type = "standard";

        BurgerFactory burgerFactory = new BurgerFactory();

        IBurger burger = burgerFactory.CreateBurger(type);

        if (burger != null)
        {
            burger.Prepare();
        }
    }
}
```

Factory Method Pattern

Instead of one factory deciding everything, **each burger type has its own factory**. It defines the interface to create an object but allows subclass to decide which class to instantiate.

```
Creator (abstract)
|
+-- ConcreteCreatorA
+-- ConcreteCreatorB
Example:
```

```
abstract class BurgerRestaurant
{
    public abstract IBurger CreateBurger();
}
Then:
```

```
class SinghBurgerRestaurant : BurgerRestaurant
{
    public override IBurger CreateBurger()
    {
        return new BasicBurger();
    }
}
```

```
class KingBurgerRestaurant : BurgerRestaurant
{
    public override IBurger CreateBurger()
    {
        return new BasicWheatBurger();
    }
}
```

Notice there is **no if-else on burger type**. The subclass itself decides the product i.e SinghBurger and KingBurger.

SinghBurger Factory
 -> BasicBurger
 -> StandardBurger
 -> PremiumBurger

KingBurger Factory
 -> BasicWheatBurger
 -> StandardWheatBurger
 -> PremiumWheatBurger

Methods in the sub create A and Create B.

Eg:
 using System;

```

// ----- Product Interface -----
public interface IBurger
{
    void Prepare();
}

// ----- Singh Burger Products -----
public class BasicBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Basic Burger with bun, patty, and ketchup!");
    }
}

public class StandardBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Standard Burger with bun, patty, cheese, and lettuce!");
    }
}

public class PremiumBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Premium Burger with gourmet bun, premium patty, cheese, lettuce, and secret sauce!");
    }
}

// ----- King Burger Products -----
public class BasicWheatBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Basic Wheat Burger with bun, patty, and ketchup!");
    }
}

public class StandardWheatBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Standard Wheat Burger with bun, patty, cheese, and lettuce!");
    }
}

public class PremiumWheatBurger : IBurger
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Premium Wheat Burger with gourmet bun, premium patty, cheese, lettuce, and secret sauce!");
    }
}

// ----- Factory Interface -----
public interface IBurgerFactory
{
    IBurger CreateBurger(string type);
}

// ----- Singh Burger Factory -----
public class SinghBurger : IBurgerFactory
{
    public IBurger CreateBurger(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicBurger();
    }
}

```

```

        if (type.Equals("standard", StringComparison.OrdinalIgnoreCase))
            return new StandardBurger();

        if (type.Equals("premium", StringComparison.OrdinalIgnoreCase))
            return new PremiumBurger();

        Console.WriteLine("Invalid burger type!");
        return null;
    }
}

// ----- King Burger Factory -----
public class KingBurger : IBurgerFactory
{
    public IBurger CreateBurger(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicWheatBurger();

        if (type.Equals("standard", StringComparison.OrdinalIgnoreCase))
            return new StandardWheatBurger();

        if (type.Equals("premium", StringComparison.OrdinalIgnoreCase))
            return new PremiumWheatBurger();

        Console.WriteLine("Invalid burger type!");
        return null;
    }
}

// ----- Main Program -----
class Program
{
    static void Main(string[] args)
    {
        string type = "basic";

        // Choose the burger family
        IBurgerFactory factory = new SinghBurger();

        // IBurgerFactory factory = new KingBurger();

        IBurger burger = factory.CreateBurger(type);

        if (burger != null)
        {
            burger.Prepare();
        }
    }
}

```

Abstract Factory :

Provide an interface for creating the families of related objects without specifying their concrete classes.

Product Family 1

```

IBurger
├── BasicBurger
├── StandardBurger
└── PremiumBurger

```

Product Family 2

```

IGarlicBread
├── BasicGarlicBread
└── CheeseGarlicBread

```

Factory Family

```

IMealFactory
├── SinghBurger

```

└─ KingBurger

Each concrete factory creates a **consistent family of products**:

SinghBurger Factory

└─ BasicBurger

└─ CheeseGarlicBread

KingBurger Factory

└─ BasicWheatBurger

└─ CheeseWheatGarlicBread

Eg:

using System;

// ----- Product 1 : Burger -----

```
public interface IBurger
```

```
{
```

```
    void Prepare();
```

```
}
```

```
public class BasicBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Basic Burger with bun, patty, and ketchup!");
```

```
    }
```

```
}
```

```
public class StandardBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Standard Burger with bun, patty, cheese, and lettuce!");
```

```
    }
```

```
}
```

```
public class PremiumBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Premium Burger with gourmet bun, premium patty, cheese, lettuce, and secret sauce!");
```

```
    }
```

```
}
```

```
public class BasicWheatBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Basic Wheat Burger with bun, patty, and ketchup!");
```

```
    }
```

```
}
```

```
public class StandardWheatBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Standard Wheat Burger with bun, patty, cheese, and lettuce!");
```

```
    }
```

```
}
```

```
public class PremiumWheatBurger : IBurger
```

```
{
```

```
    public void Prepare()
```

```
    {
```

```
        Console.WriteLine("Preparing Premium Wheat Burger with gourmet bun, premium patty, cheese, lettuce, and secret sauce!");
```

```
    }
```

```
}
```

// ----- Product 2 : Garlic Bread -----

```

public interface IGarlicBread
{
    void Prepare();
}

public class BasicGarlicBread : IGarlicBread
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Basic Garlic Bread with butter and garlic!");
    }
}

public class CheeseGarlicBread : IGarlicBread
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Cheese Garlic Bread with extra cheese and butter!");
    }
}

public class BasicWheatGarlicBread : IGarlicBread
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Basic Wheat Garlic Bread with butter and garlic!");
    }
}

public class CheeseWheatGarlicBread : IGarlicBread
{
    public void Prepare()
    {
        Console.WriteLine("Preparing Cheese Wheat Garlic Bread with extra cheese and butter!");
    }
}

// ----- Abstract Factory -----

public interface IMealFactory
{
    IBurger CreateBurger(string type);
    IGarlicBread CreateGarlicBread(string type);
}

// ----- Concrete Factory : Singh Burger -----

public class SinghBurger : IMealFactory
{
    public IBurger CreateBurger(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicBurger();

        if (type.Equals("standard", StringComparison.OrdinalIgnoreCase))
            return new StandardBurger();

        if (type.Equals("premium", StringComparison.OrdinalIgnoreCase))
            return new PremiumBurger();

        Console.WriteLine("Invalid Burger Type!");
        return null;
    }

    public IGarlicBread CreateGarlicBread(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicGarlicBread();

        if (type.Equals("cheese", StringComparison.OrdinalIgnoreCase))

```

```

        return new CheeseGarlicBread();

        Console.WriteLine("Invalid Garlic Bread Type!");
        return null;
    }
}

// ----- Concrete Factory : King Burger -----

public class KingBurger : IMealFactory
{
    public IBurger CreateBurger(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicWheatBurger();

        if (type.Equals("standard", StringComparison.OrdinalIgnoreCase))
            return new StandardWheatBurger();

        if (type.Equals("premium", StringComparison.OrdinalIgnoreCase))
            return new PremiumWheatBurger();

        Console.WriteLine("Invalid Burger Type!");
        return null;
    }

    public IGarlicBread CreateGarlicBread(string type)
    {
        if (type.Equals("basic", StringComparison.OrdinalIgnoreCase))
            return new BasicWheatGarlicBread();

        if (type.Equals("cheese", StringComparison.OrdinalIgnoreCase))
            return new CheeseWheatGarlicBread();

        Console.WriteLine("Invalid Garlic Bread Type!");
        return null;
    }
}

// ----- Client -----

class Program
{
    static void Main(string[] args)
    {
        string burgerType = "basic";
        string garlicBreadType = "cheese";

        IMealFactory mealFactory = new SinghBurger();

        // IMealFactory mealFactory = new KingBurger();

        IBurger burger = mealFactory.CreateBurger(burgerType);
        IGarlicBread garlicBread = mealFactory.CreateGarlicBread(garlicBreadType);

        burger?.Prepare();
        garlicBread?.Prepare();
    }
}

```